

Part 1: Debugging Flaky Test Code (30 minutes)

A previous intern wrote this Playwright test for user login functionality. **The test is flaky** - sometimes passes, sometimes fails in CI/CD pipeline.

```
import pytest
from playwright.sync_api import sync_playwright

def test_user_login():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        # Navigate to login page
        page.goto("https://app.workflowpro.com/login")

        # Fill login form
        page.fill("#email", "admin@company1.com")
        page.fill("#password", "password123")
        page.click("#login-btn")

        # Verify successful login
        assert page.url == "https://app.workflowpro.com/dashboard"
        assert page.locator(".welcome-message").is_visible()

        browser.close()

def test_multi_tenant_access():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        page.goto("https://app.workflowpro.com/login")
        page.fill("#email", "user@company2.com")
        page.fill("#password", "password123")
        page.click("#login-btn")

        # User should only see Company2 data
        projects = page.locator(".project-card").all()
        for project in projects:
            assert "Company2" in project.text_content()

        browser.close()
```

Your Tasks:

1. **Identify Flakiness Issues:** List all problems that could cause intermittent failures
2. **Explain Root Causes:** Why do these issues happen in CI/CD vs local environment?
3. **Provide Fixes:** Write the corrected version with proper waits, error handling, and reliability improvements

Additional Context (you may need to ask for more):

- App uses dynamic loading for dashboard elements
- Login process includes 2FA for some users
- Different tenants have different loading times
- CI runs on different browsers and screen sizes

Answer :

1. Objective

The objective of this task is to analyze the existing Playwright automation tests, identify the causes of flaky behavior, explain why these failures occur more frequently in Continuous Integration and Continuous Deployment (CI/CD) environments than on local machines, and propose reliable improvements following Playwright automation best practices.

The application under test is a multi-tenant B2B SaaS platform that supports multiple organizations, different user roles, browser compatibility, mobile devices, and third-party integrations. Such applications require stable and maintainable automation tests that can execute consistently across different environments. Therefore, the focus of this task is not only to fix the existing code but also to improve the overall reliability and maintainability of the test suite.

2. Analysis of the Existing Test Cases

The provided Playwright scripts validate two important business scenarios.

The first test verifies whether a user can successfully log in to the application by navigating to the login page, entering valid credentials, clicking the login button, and checking whether the dashboard is displayed successfully.

The second test validates tenant isolation by logging in as a user belonging to Company 2 and verifying that only Company 2 projects are visible. Since the application follows a multi-tenant architecture, ensuring proper tenant isolation is a critical security requirement.

Although both tests correctly represent the intended business functionality, several implementation issues make them unreliable. These issues can result in intermittent failures, particularly when executed in automated CI/CD pipelines or cloud-based testing environments.

3. Flakiness Issues Identified

3.1 Missing Synchronization After Login

One of the primary causes of flakiness is the absence of synchronization after clicking the login button.

The current implementation immediately validates the dashboard URL after clicking the login button.

```
page.click("#login-btn")
```

```
assert page.url == "https://app.workflowpro.com/dashboard"
```

This implementation assumes that authentication, navigation, and dashboard loading happen instantly. In reality, clicking the login button only starts the authentication process. The application first sends an authentication request to the server, validates the user's credentials, generates an authentication token, redirects the user to the dashboard, and finally loads dashboard information through multiple API requests.

Since all these operations happen asynchronously, there is no guarantee that the dashboard has finished loading before the assertion is executed.

On a developer's local machine, where the browser and server usually run with minimal delay, this issue may not appear frequently. However, in CI/CD environments where machines are shared and network latency is higher, navigation generally takes longer, causing the assertion to execute before the page has completely loaded.

A more reliable approach is to explicitly wait until the browser finishes navigation before validating the dashboard.

```
with page.expect_navigation():
```

```
    page.click("#login-btn")
```

Alternatively, Playwright provides URL-based waiting.

```
page.click("#login-btn")
```

```
page.wait_for_url("**/dashboard")
```

Using explicit waits ensures that validation begins only after the browser has successfully completed navigation.

3.2 Dynamic Loading of Dashboard Elements

The assessment specifies that dashboard elements are loaded dynamically. This means that although the dashboard page may have opened successfully, important components such as the welcome message, project list, notifications, and analytics widgets are still being fetched from backend APIs.

The existing implementation immediately verifies the visibility of the welcome message.

```
assert page.locator(".welcome-message").is_visible()
```

If the application has not yet completed rendering the dashboard, the locator may either not exist or remain hidden temporarily, resulting in intermittent test failures.

Playwright provides built-in assertions that automatically wait until the expected condition is satisfied. Instead of checking visibility immediately, the automation should wait until the welcome message becomes visible.

```
from playwright.sync_api import expect
```

```
expect(page.locator(".welcome-message")).to_be_visible(timeout=15000)
```

This approach significantly improves test stability because Playwright continuously checks the element until the specified timeout is reached.

3.3 Exact URL Comparison

The current implementation compares the complete dashboard URL using an equality check.

```
assert page.url == "https://app.workflowpro.com/dashboard"
```

Although this appears correct, modern web applications often append query parameters, fragments, or trailing slashes to URLs.

For example, the following URLs may all represent the same dashboard.

```
https://app.workflowpro.com/dashboard
```

```
https://app.workflowpro.com/dashboard/
```

```
https://app.workflowpro.com/dashboard?welcome=true
```

```
https://app.workflowpro.com/dashboard#home
```

An exact string comparison would incorrectly fail even though the user has successfully logged in.

A better solution is to verify only the important portion of the URL.

```
import re
```

```
expect(page).to_have_url(re.compile(".*dashboard"))
```

This approach accommodates minor URL variations while still validating successful navigation.

3.4 Absence of Timeout Configuration

Different tenants in a multi-tenant SaaS application may contain different volumes of business data.

For example, one tenant may have only a few projects, while another may have hundreds of projects, reports, dashboards, notifications, and team members.

As a result, dashboard loading time varies significantly between tenants.

The current test relies entirely on Playwright's default timeout settings, which may not always be sufficient for larger tenants or slower CI/CD environments.

To improve stability, the automation framework should define a reasonable timeout that accommodates slower loading scenarios.

```
page.set_default_timeout(15000)
```

Additionally, important assertions should define their own timeout values whenever required.

```
expect(locator).to_be_visible(timeout=15000)
```

Configurable timeout values make the framework more adaptable to different environments.

3.5 Two-Factor Authentication (2FA) Is Not Considered

The assessment clearly states that some users require two-factor authentication during login.

The current automation assumes that entering the username and password always redirects the user directly to the dashboard.

However, the actual authentication flow may include an intermediate OTP verification screen before granting access.

If the automation account requires OTP verification, the test will fail because it never handles the additional authentication step.

A practical solution is to use a dedicated automation account with two-factor authentication disabled. If bypassing 2FA is not possible, the automation framework should detect the OTP page and complete the verification process before continuing.

```
if page.locator("#otp").is_visible(timeout=3000):
```

```
    page.fill("#otp", otp_code)
```

```
    page.click("#verify")
```

Handling multiple authentication flows ensures that the automation remains stable for different user configurations.

3.6 Hardcoded Credentials

The provided test script stores login credentials directly inside the source code.

```
page.fill("#email", "admin@company1.com")
```

```
page.fill("#password", "password123")
```

Hardcoding credentials introduces several problems. It exposes sensitive information, increases maintenance effort whenever credentials change, and prevents the same test from being executed across multiple environments.

A more secure approach is to retrieve credentials from environment variables or a secure secrets management service.

```
import os
```

```
EMAIL = os.getenv("EMAIL")
```

```
PASSWORD = os.getenv("PASSWORD")
```

This approach improves security while allowing the same automation suite to execute in development, QA, staging, and production environments without code modifications.

3.7 Inconsistent Browser Configuration

The current implementation launches the browser using default settings without specifying viewport dimensions.

```
browser = p.chromium.launch()
```

Responsive web applications often display different layouts depending on screen resolution.

A desktop layout may display a full navigation menu, while a smaller viewport may collapse the same menu into a hamburger icon. Such differences can cause element locators to behave differently across browsers and devices.

To maintain consistent execution, the browser context should define a fixed viewport.

```
context = browser.new_context(
```

```
viewport={  
  "width": 1440,  
  "height": 900  
}  
)
```

A standardized browser configuration ensures consistent rendering across different execution environments.

3.8 Immediate Retrieval of Project Cards

The tenant validation test retrieves all project cards immediately after login.

```
projects = page.locator(".project-card").all()
```

The `.all()` method returns only the elements currently present in the Document Object Model (DOM). If project cards are still loading, the method may return an empty collection.

As a result, the verification loop executes zero times, causing the test to pass incorrectly even though project data has not loaded.

A better implementation first waits for the project cards to appear before counting and validating them.

```
cards = page.locator(".project-card")
```

```
expect(cards.first()).to_be_visible()
```

```
count = cards.count()
```

```
for index in range(count):
```

```
    expect(cards.nth(index)).to_contain_text("Company2")
```

This approach ensures that the tenant verification begins only after project data has finished loading.

3.9 Lack of Test Isolation

Each test launches a new browser instance but does not isolate browser sessions or clean up application data after execution.

Residual cookies, cached authentication tokens, or previously created test data may influence subsequent test executions, leading to inconsistent behavior.

A better solution is to use Pytest fixtures that automatically perform setup before each test and cleanup afterward.

This guarantees that every test starts with a clean browser session and executes independently of previous tests.

3.10 Absence of Retry Strategy

Temporary infrastructure issues such as network latency, BrowserStack connectivity problems, or short-lived backend outages may occasionally cause tests to fail even when the application itself is functioning correctly.

Although retries should never hide genuine software defects, selectively retrying transient infrastructure failures can significantly improve pipeline stability.

A retry mechanism should therefore be configured only for known temporary failures while ensuring that genuine application defects remain visible.

4. Root Causes of Flaky Tests

After identifying the issues in the existing automation scripts, it is equally important to understand why these problems occur. Most flaky tests are not caused by application defects but by poor synchronization, unstable test design, improper test data management, or environmental differences. Understanding these root causes helps in designing automation that is reliable, maintainable, and suitable for continuous integration pipelines.

4.1 Asynchronous Application Behavior

Modern web applications do not load all information immediately after navigation. Instead, they load data asynchronously by making multiple API requests in the background. In the given B2B SaaS application, the login process does not end immediately after clicking the login button. After authentication, the application redirects the user to the dashboard, where additional API calls retrieve projects, notifications, user details, and tenant-specific information.

If the automation attempts to verify page elements before these background operations have completed, the assertions execute too early and fail intermittently. Since the loading time varies depending on network speed and server response time, the failures appear random, making the tests flaky.

To prevent this issue, the automation should always wait for navigation and dynamically loaded elements before performing validations.

4.2 Differences Between Local and CI/CD Environments

One of the primary reasons flaky tests appear during pipeline execution but not on a developer's local machine is the difference in execution environments.

Local systems generally have dedicated hardware resources, faster processors, stable internet connections, and fewer concurrent processes. Consequently, pages load quickly, and automation steps execute smoothly.

In contrast, CI/CD pipelines typically run on shared virtual machines or containers where CPU resources, memory, and network bandwidth are shared among multiple jobs. These limitations increase application loading times and expose synchronization problems that remain hidden during local execution.

Furthermore, CI/CD pipelines usually execute tests across multiple browsers and operating systems simultaneously. Variations in browser rendering engines, JavaScript execution speed, and network latency increase the likelihood of timing-related failures.

Therefore, automation tests should never rely on assumptions about execution speed. Instead, they should always synchronize with actual application behavior.

4.3 Dynamic Data Loading

The assessment specifies that different tenants have different loading times. This is expected in a multi-tenant SaaS application because each organization maintains its own data.

For example, Company A may contain only a few projects, while Company B may contain hundreds of active projects, archived projects, reports, and dashboards. As the amount of tenant-specific data increases, the time required to retrieve and render information also increases.

If the automation immediately validates project cards after login, larger tenants may still be loading data, resulting in intermittent failures.

To improve reliability, the framework should wait until project cards become visible before validating their contents.

4.4 Browser Compatibility Differences

The assessment states that tests execute across Chrome, Firefox, Safari, and mobile devices.

Although Playwright provides consistent automation APIs, different browsers use different rendering engines.

- Chrome uses the Blink rendering engine.
- Firefox uses the Gecko rendering engine.

- Safari uses the WebKit rendering engine.

Each engine handles rendering, JavaScript execution, animations, and network requests slightly differently. As a result, a test that passes consistently in Chrome may occasionally fail in Firefox or Safari if synchronization is not implemented correctly.

To reduce browser-specific issues, the framework should avoid fixed delays and instead rely on Playwright's built-in waiting mechanisms.

4.5 Responsive User Interface

The application supports desktop browsers and mobile devices. Responsive web applications often change their layout depending on screen size.

For example, a navigation menu displayed horizontally on a desktop screen may be collapsed into a hamburger menu on a mobile device. Buttons, forms, and project cards may also change position depending on viewport dimensions.

If viewport size is not standardized, automation scripts may behave differently on different devices, leading to inconsistent results.

A fixed browser context should therefore be configured during execution, while separate tests should validate responsive behavior across multiple resolutions.

4.6 Test Data Dependency

The current implementation assumes that the same users, projects, and tenant data always exist in the testing environment.

In practice, test environments change frequently. Users may be deleted, passwords may expire, projects may be modified, or databases may be refreshed.

Automation that depends on static data is therefore difficult to maintain and often fails unexpectedly.

A better approach is to create the required test data dynamically before execution and remove it after the test completes. This ensures that every test starts with a known and consistent state.

4.7 Shared Test Environment

Most enterprise applications use shared QA or staging environments where multiple testers and automation suites execute simultaneously.

Suppose two automation jobs attempt to create a project named "Test Project" at the same time. One test may fail because the project already exists, while another may accidentally validate the wrong project.

To prevent such conflicts, automation should generate unique test data using timestamps or universally unique identifiers (UUIDs).

For example, project names can follow the format:

Test_Project_20260727_153000

or

Test_Project_f83b91

Generating unique test data prevents collisions during parallel execution.

4.8 Lack of Proper Error Handling

The current implementation assumes that every operation succeeds. If navigation fails due to a temporary network issue or an API returns an unexpected response, the test terminates immediately without providing useful diagnostic information.

Reliable automation should include appropriate error handling, screenshots, execution logs, Playwright traces, and browser recordings whenever a failure occurs. These artifacts significantly reduce debugging time and help identify the exact cause of failures.

5. Corrected Playwright Implementation

The following implementation incorporates the improvements discussed above and follows Playwright best practices to improve reliability and maintainability.

```
import os

import re

from playwright.sync_api import sync_playwright, expect

def test_user_login():

    with sync_playwright() as p:

        browser = p.chromium.launch(
            headless=True
```

```
)
```

```
context = browser.new_context(
```

```
    viewport={
```

```
        "width": 1440,
```

```
        "height": 900
```

```
    }
```

```
)
```

```
page = context.new_page()
```

```
page.set_default_timeout(15000)
```

```
page.goto(
```

```
    "https://app.workflowpro.com/login",
```

```
    wait_until="networkidle"
```

```
)
```

```
page.fill("#email", os.getenv("EMAIL"))
```

```
page.fill("#password", os.getenv("PASSWORD"))
```

```
with page.expect_navigation():
```

```
    page.click("#login-btn")
```

```
# Optional 2FA Handling
```

```
if page.locator("#otp").is_visible(timeout=3000):
```

```
page.fill("#otp", os.getenv("OTP"))

page.click("#verify")


expect(page).to_have_url(

    re.compile(".*dashboard")

)


expect(

    page.locator(".welcome-message")

).to_be_visible(timeout=15000)


browser.close()
```

Explanation of Improvements

The updated implementation introduces several improvements that enhance the reliability of the automation.

Instead of immediately validating the dashboard after clicking the login button, the test explicitly waits for navigation to complete. This prevents assertions from executing before the application finishes redirecting the user.

Sensitive information such as usernames and passwords has been removed from the source code and replaced with environment variables. This improves security and allows the same automation suite to execute across multiple environments without modification.

The browser context now uses a fixed viewport size, ensuring consistent rendering across different execution environments. Additionally, the browser executes in headless mode, matching the configuration commonly used in CI/CD pipelines.

Playwright's built-in `expect()` assertions are used instead of immediate assertions. These assertions automatically wait until the expected condition becomes true, significantly reducing flaky failures caused by asynchronous page loading.

The implementation also includes optional support for two-factor authentication by detecting the OTP page and completing verification when necessary.

Overall, the revised implementation follows Playwright best practices and provides a much more stable and maintainable automation solution.

6. Additional Reliability Improvements

While the corrected implementation significantly improves test stability, several additional enhancements would be implemented in a production-grade automation framework.

The first improvement would be adopting the **Page Object Model (POM)** design pattern. Separating page-specific logic from test scripts improves readability, promotes code reuse, and reduces maintenance effort whenever the user interface changes.

Pytest fixtures would be introduced to handle browser initialization, authentication, test setup, and cleanup. This ensures that each test begins with a clean execution environment and eliminates duplicated setup code.

Environment-specific configuration files would be maintained for Development, QA, Staging, and Production environments. These files would store application URLs, timeout values, tenant information, and browser configurations, allowing the same test suite to execute across multiple environments without code changes.

Secure credential management would replace hardcoded usernames and passwords. Sensitive information would be stored in environment variables or enterprise secrets management solutions such as GitHub Secrets, Azure Key Vault, or AWS Secrets Manager.

Automatic screenshots, browser traces, execution videos, and application logs would be captured whenever a test fails. These artifacts significantly simplify debugging and provide valuable information for failure analysis.

To improve execution speed, test data creation and cleanup would be performed using backend APIs instead of the user interface. API-based setup is significantly faster and reduces dependency on UI interactions.

Finally, the framework would integrate reporting tools such as Allure Reports or HTML Reports into the CI/CD pipeline, enabling stakeholders to review execution results, failure trends, screenshots, and logs after every test run.

7. Assumptions

The following assumptions have been made while analyzing and improving the provided automation scripts because certain implementation details were not specified in the assessment.

- A dedicated automation account is available for testing.
- The automation environment allows API-based authentication and test data creation.
- Stable element locators are available for all user interface components.
- Test environments permit automatic cleanup of created projects after execution.
- BrowserStack credentials are securely configured through environment variables.
- Two-factor authentication can either be bypassed for automation users or completed using a predefined OTP mechanism.

- Each tenant maintains isolated project data, allowing tenant-specific validation.
- The application uses asynchronous API calls to populate dashboard information after login.

8. Conclusion

The original Playwright implementation demonstrates the intended business flow but contains several synchronization and design issues that make it unreliable in real-world automation environments. Most failures originate from asynchronous page loading, dynamic content rendering, hardcoded configuration values, and inadequate synchronization.

By introducing explicit waits, Playwright's built-in assertions, environment-based configuration management, secure credential handling, proper browser configuration, and improved test isolation, the automation becomes significantly more stable across local execution, CI/CD pipelines, multiple browsers, and different tenant environments.

These improvements not only reduce flaky failures but also create a strong foundation for building a scalable enterprise automation framework capable of supporting cross-browser, cross-platform, and multi-tenant testing.

Part 2: Test Framework Design (25 minutes)

Design a test automation framework structure for this B2B SaaS platform. **Note:** Requirements are intentionally incomplete.

Given Requirements:

- Support web (Chrome, Firefox, Safari) and mobile (iOS, Android) testing
- Handle multiple tenant environments (company1.workflowpro.com, company2.workflowpro.com)
- Test different user roles (Admin, Manager, Employee) with varying permissions
- API testing for backend services
- Integration with BrowserStack for cross-platform testing
- CI/CD pipeline integration

Your Tasks:

1. **Framework Structure:** Design folder structure, base classes, and utilities

2. **Configuration Management:** How to handle multiple environments, browsers, and test data
3. **Identify Missing Requirements:** What questions would you ask about test data management, reporting, parallel execution, etc.?

Format: Use text/pseudocode/diagrams to explain your design

Answer :

1. Objective

The objective of this task is to design a scalable, maintainable, and reusable test automation framework for the **WorkFlow Pro** B2B SaaS platform. The framework should support web automation across multiple browsers, mobile testing, API testing, multi-tenant environments, different user roles, BrowserStack integration, and CI/CD execution.

Since this is an enterprise SaaS application serving multiple organizations, the framework should be modular, easy to maintain, and capable of handling future feature additions with minimal code changes. A well-designed framework not only reduces maintenance effort but also improves test execution speed, code reusability, and overall reliability.

2. Framework Design Approach

For this application, I would choose a **Hybrid Test Automation Framework** combining the following design patterns and technologies:

- **Page Object Model (POM)** for UI automation.
- **Pytest** as the test execution framework.
- **Playwright** for browser automation.
- **Requests** library for API testing.
- **BrowserStack** for cross-browser and mobile testing.
- **Pytest Fixtures** for setup and teardown.
- **Configuration-based execution** for multiple environments.
- **Data-driven testing** for handling different tenants, users, and browsers.

This hybrid approach separates business logic, page interactions, test data, and configuration files, making the framework scalable and easy to maintain.

3. Proposed Framework Architecture

I would organize the project into the following directory structure.

workflowpro-automation/

|

|— tests/

| |— web/

| |— api/

| |— mobile/

| |— integration/

|

|— pages/

| |— login_page.py

| |— dashboard_page.py

| |— project_page.py

|

|— api/

| |— auth_api.py

| |— project_api.py

| |— tenant_api.py

|

|— fixtures/

| |— browser_fixture.py

| |— api_fixture.py

| |— auth_fixture.py

|

|— utils/

| |— logger.py

```
| |— helpers.py
| |— browser_factory.py
| |— data_generator.py
|
|— config/
| |— dev.yaml
| |— qa.yaml
| |— staging.yaml
| |— production.yaml
|
|— test_data/
|
|— browserstack/
|
|— reports/
|
|— screenshots/
|
|— conftest.py
|
|— pytest.ini
|
|— requirements.txt
```

This structure separates different responsibilities into dedicated modules, making the framework easier to understand and maintain.

4. Page Object Model (POM)

One of the most important design decisions is the use of the **Page Object Model**.

Instead of placing Playwright commands directly inside test cases, each web page is represented as a Python class containing all page-specific locators and methods.

For example, instead of writing the following code in every test,

```
page.fill("#email", email)
```

```
page.fill("#password", password)
```

```
page.click("#login-btn")
```

the login functionality can be encapsulated inside a dedicated LoginPage class.

```
class LoginPage:
```

```
    def login(self, email, password):
```

```
        self.page.fill("#email", email)
```

```
        self.page.fill("#password", password)
```

```
        self.page.click("#login-btn")
```

The test then becomes much cleaner.

```
login_page.login(email, password)
```

This approach offers several advantages:

- UI changes only require updates in one place.
- Tests become shorter and easier to read.
- Page interactions become reusable across multiple test cases.
- Maintenance effort is significantly reduced.

5. Test Layer Separation

To improve maintainability, I would separate the framework into different testing layers.

The **Web Layer** would contain Playwright tests responsible for validating user interface functionality such as login, project creation, project updates, dashboards, reports, and permissions.

The **API Layer** would contain backend API validations using the Requests library. These tests would verify authentication, project management APIs, tenant APIs, and other backend services.

The **Mobile Layer** would contain BrowserStack-based tests for Android and iOS devices to validate responsive behavior and mobile compatibility.

The **Integration Layer** would combine API and UI testing. For example, a project can be created using an API call and then verified through both the web application and mobile application.

Separating these layers improves organization and allows each type of testing to evolve independently.

6. Configuration Management

The application supports multiple environments and multiple tenants.

Instead of hardcoding URLs or credentials inside test scripts, all configuration values should be stored in external configuration files.

For example,

Development

base_url: https://dev.workflowpro.com

browser: chromium

timeout: 15000

QA

base_url: https://qa.workflowpro.com

browser: chromium

timeout: 15000

Each environment can have different URLs, credentials, timeout values, BrowserStack configurations, and API endpoints.

During execution, the framework should load the required configuration automatically.

Example command:

```
pytest --env=qa
```

This allows the same automation suite to execute against different environments without modifying the source code.

7. Browser Configuration

The framework must support execution on Chrome, Firefox, Safari, and mobile browsers.

Instead of writing separate code for each browser, browser selection should be handled dynamically.

Example:

```
pytest --browser=chromium
```

```
pytest --browser=firefox
```

```
pytest --browser=webkit
```

A Browser Factory class would initialize the appropriate browser based on the execution parameters.

This design improves flexibility while keeping the test scripts browser-independent.

8. Multi-Tenant Configuration

Since WorkFlow Pro is a multi-tenant SaaS platform, different organizations use separate application instances.

Examples include:

```
company1.workflowpro.com
```

```
company2.workflowpro.com
```

```
company3.workflowpro.com
```

Each tenant may have:

- Different users
- Different permissions
- Different projects
- Different loading times
- Different business data

The framework should therefore store tenant information separately.

For example,

```
tenant: company1
```

base_url: https://company1.workflowpro.com

admin_email: admin@company1.com

Changing tenants should require only a configuration update rather than modifications to the test scripts.

9. User Role Management

The assessment specifies three user roles:

- Administrator
- Manager
- Employee

Each role has different permissions.

Instead of creating separate login scripts for each role, the framework should provide a centralized authentication utility.

Example:

```
login_as_admin()
```

```
login_as_manager()
```

```
login_as_employee()
```

Each function retrieves the appropriate credentials from the configuration file and authenticates the corresponding user.

This approach improves readability while reducing duplicate code.

10. Test Data Management

One of the biggest challenges in enterprise automation is managing test data.

Using static test data often causes failures because projects, users, or other entities may already exist from previous executions.

Instead, the framework should generate unique data during execution.

For example,

```
project_name = "Project_" + timestamp
```

or

```
project_name = uuid.uuid4()
```

Using dynamically generated data prevents conflicts during parallel execution.

Whenever possible, test data should be created using backend APIs instead of the user interface because API-based setup is significantly faster and more reliable.

After the test completes, all created data should be removed automatically to maintain a clean testing environment.

11. API Utility Layer

Since the framework includes API testing, API-related functionality should be encapsulated into reusable utility classes.

For example,

```
ProjectAPI.create_project()
```

```
ProjectAPI.delete_project()
```

```
ProjectAPI.get_project()
```

```
TenantAPI.get_users()
```

This approach allows API functionality to be reused across multiple test scenarios, reducing code duplication and improving maintainability.

12. BrowserStack Integration

BrowserStack plays an important role in validating browser compatibility and mobile responsiveness.

Instead of maintaining physical devices, BrowserStack provides access to a wide range of browsers, operating systems, and mobile devices.

The framework should integrate BrowserStack through configuration files.

For example,

browserstack_username:

browserstack_access_key:

platform:

browser:

device:

Tests should execute locally during development and automatically switch to BrowserStack during CI/CD execution.

This minimizes cloud testing costs while ensuring sufficient browser coverage before releases.

13. Logging and Reporting

Every automation framework should provide meaningful execution reports.

The framework should generate:

- Execution logs
- Screenshots on failure
- Playwright traces
- Browser videos
- HTML Reports
- Allure Reports

These artifacts make debugging significantly easier and provide detailed execution history for developers and QA engineers.

Reports should be automatically published after every CI/CD pipeline execution.

14. CI/CD Pipeline Integration

The automation framework should integrate seamlessly with modern CI/CD tools such as GitHub Actions, Azure DevOps, or Jenkins.

A typical pipeline would execute the following steps:

1. Clone the latest repository.
2. Install Python dependencies.
3. Install Playwright browsers.
4. Load environment variables and secrets.
5. Execute API tests.
6. Execute web automation tests.
7. Execute BrowserStack cross-browser tests.
8. Generate execution reports.
9. Publish reports and logs.
10. Notify the development team of execution results.

Running API tests before UI tests helps identify backend failures early, reducing unnecessary UI execution.

15. Parallel Test Execution

To reduce execution time, the framework should support parallel execution using Pytest.

Since UI tests are generally slower than API tests, running multiple tests simultaneously significantly reduces overall execution time.

However, parallel execution introduces challenges such as shared test data and environment conflicts.

To avoid these issues, every test should generate unique data, use isolated browser sessions, and perform cleanup after execution.

16. Security Considerations

The framework should follow secure automation practices.

Sensitive information such as usernames, passwords, API tokens, BrowserStack credentials, and authentication keys should never be stored directly in source code.

Instead, these values should be retrieved from secure storage such as environment variables or enterprise secrets management solutions.

This approach improves security while allowing the same framework to execute across multiple environments.

17. Missing Requirements and Clarifications

Although the assessment provides several requirements, some important implementation details are missing. Before designing the complete framework, I would seek clarification on the following points:

Test Data Management

- Will dedicated test environments be available?
- Can automation create and delete test data using APIs?
- Will databases be reset regularly?
- Can multiple automation suites execute simultaneously?

Authentication

- Is Single Sign-On (SSO) supported?
- Can automation accounts bypass two-factor authentication?
- How are API tokens generated and refreshed?

Tenant Management

- How many tenant environments need to be tested?
- Can automation switch tenants dynamically?
- Are tenant databases completely isolated?

BrowserStack

- What is the available BrowserStack concurrency limit?
- Which browser and device combinations are considered mandatory?
- Should visual testing also be performed?

Reporting

- Which reporting framework is preferred?
- Should reports be integrated with Jira or TestRail?
- Are screenshots and videos required for every failure?

CI/CD

- Which CI/CD platform is currently used?
- How frequently will automation execute?
- Should smoke tests and regression tests run separately?

Clarifying these questions would help design a framework that aligns with the organization's testing strategy and infrastructure.

18. Conclusion

The proposed automation framework follows a modular and scalable architecture suitable for an enterprise B2B SaaS application. By combining the Page Object Model, Pytest, Playwright, API utilities, BrowserStack integration, centralized configuration management, and secure test data handling, the framework becomes easy to maintain, reusable, and adaptable to future requirements.

The design also supports multiple tenants, multiple browsers, mobile devices, user roles, and CI/CD pipelines while minimizing code duplication and improving execution reliability. This architecture provides a strong foundation for implementing efficient automated testing across the entire WorkFlow Pro platform.

Part 3: API + UI Integration Test (35 minutes)

Create a test that validates the complete flow of project creation through both API and UI layers.

Business Scenario:

1. Create a project via API
2. Verify project appears correctly in web UI
3. Check project is accessible on mobile
4. Validate proper tenant isolation (project only visible to correct company)

API Endpoint Information:

POST /api/v1/projects

Headers: Authorization: Bearer {token}, X-Tenant-ID: {company_id}

Body: {"name": "Test Project", "description": "...", "team_members": [...]}

Response: {"id": 123, "name": "Test Project", "status": "active"}

Your Tasks:

1. **Write Integration Test:** Combine pytest (API) + Playwright (UI) + BrowserStack mobile testing
2. **Handle Test Data:** Show how you'd manage test data across API/UI
3. **Cross-Platform Validation:** Ensure the test works on multiple browsers and mobile devices
4. **Tenant Isolation:** Verify security boundaries between companies

Expected Test Structure:

```
def test_project_creation_flow():  
    # 1. API: Create project  
    # 2. Web UI: Verify project display  
    # 3. Mobile: Check mobile accessibility  
    # 4. Security: Verify tenant isolation  
    pass
```

Your Tasks:

1. **Implement the test** using pytest + Playwright + BrowserStack concepts
2. **Handle edge cases** (network failures, slow loading, mobile responsiveness)
3. **Explain approach** with comments on your testing strategy

Hints: You'll need to make assumptions about authentication, test data cleanup, and mobile testing setup.

Answer :

1. Objective

The objective of this task is to design and implement an end-to-end integration test that validates the complete project creation workflow across multiple layers of the application. Instead of testing the API and User Interface independently, this approach verifies that all components of the application work together correctly.

The workflow begins by creating a project through the backend API. The created project is then validated through the web user interface, followed by verification on a mobile device using BrowserStack. Finally, tenant isolation is validated to ensure that the project is only accessible to the tenant that created it and remains invisible to other organizations.

This testing approach provides greater confidence than isolated API or UI tests because it verifies the complete business workflow from backend services to the end-user experience.

2. Testing Strategy

Rather than creating the project through the user interface, the project will first be created using the API.

There are several advantages to this approach.

Creating test data through APIs is significantly faster than performing the same actions through the UI. API-based setup also reduces dependency on the frontend, making the tests less susceptible to failures caused by UI changes. Furthermore, it allows the test to focus only on validating that the project is displayed correctly instead of spending time interacting with multiple UI forms.

Once the project is successfully created, the automation logs into the web application using Playwright and verifies that the newly created project appears in the dashboard.

After successful web validation, the same project is verified on a mobile device using BrowserStack to ensure that the project is accessible across different platforms.

Finally, the test logs into another tenant and verifies that the created project is not visible, confirming proper tenant isolation.

After all validations are completed, the project is deleted through the API to maintain a clean testing environment.

3. Business Workflow

The complete business workflow implemented by the integration test is shown below.

Create Project (API)

|



Verify API Response

|



Login to Web Application

|



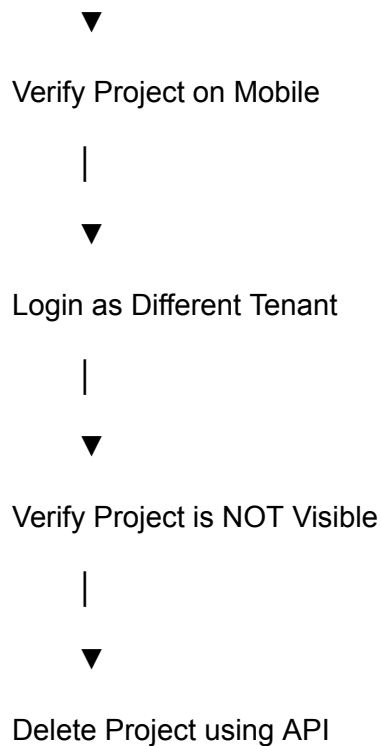
Verify Project Appears in Dashboard

|



Open Mobile Device (BrowserStack)

|



This workflow validates both the functional correctness and security of the application.

4. Test Data Management Strategy

One of the biggest challenges in automation is managing test data.

If every test creates a project named "Test Project", multiple parallel executions may interfere with each other.

To prevent conflicts, every execution should generate a unique project name.

For example,

import uuid

```
project_name = f"Automation_Project_{uuid.uuid4().hex[:8]}"
```

Using UUIDs guarantees uniqueness across parallel executions and eliminates naming collisions.

Instead of manually creating users or projects before execution, the required test data should be created dynamically through API calls.

Once the test completes successfully, all created data should be deleted automatically.

This approach ensures that every test begins with a clean environment and does not affect subsequent executions.

5. Authentication Strategy

Both API and UI testing require authentication.

For API requests, authentication tokens should be generated before execution and included in the request headers.

Example:

```
headers = {  
    "Authorization": f"Bearer {token}",  
    "X-Tenant-ID": tenant_id  
}
```

For UI automation, Playwright logs into the application using dedicated automation credentials stored securely in environment variables.

This ensures that sensitive information is never stored directly in the source code.

6. API Implementation

The first step of the workflow is creating a new project through the backend API.

```
import requests
```

```
response = requests.post(  
    url="https://app.workflowpro.com/api/v1/projects",  
    headers=headers,  
    json={  
        "name": project_name,  
        "description": "Created through automation",  
        "team_members": []  
    }  
)
```

```
)
```

```
assert response.status_code == 201
```

```
project_id = response.json()["id"]
```

The test first verifies that the server returns a successful response.

It then extracts the project ID from the response, which will be used later for cleanup.

Performing setup through the API significantly reduces execution time compared to creating the project manually through the user interface.

7. Web UI Validation Using Playwright

Once the project has been created successfully through the API, the automation validates that the project appears in the web application.

```
from playwright.sync_api import expect
```

```
page.goto("https://app.workflowpro.com/login")
```

```
page.fill("#email", EMAIL)
```

```
page.fill("#password", PASSWORD)
```

```
with page.expect_navigation():
```

```
    page.click("#login-btn")
```

```
page.reload()
```

```
expect(
```

```
    page.locator(f"text={project_name}")
```



```
).to_be_visible(timeout=15000)
```

The dashboard is refreshed after login to ensure that the latest project list is retrieved from the backend.

Playwright's built-in assertions automatically wait until the project becomes visible.

This synchronization significantly reduces flaky failures.

8. Mobile Validation Using BrowserStack

After successful web validation, the same project is verified on a mobile device.

Instead of maintaining physical Android and iOS devices, BrowserStack provides cloud-hosted real devices and browsers.

The same Playwright script can be executed against BrowserStack by changing the execution configuration.

Example BrowserStack capabilities:

```
capabilities = {  
    "browserName": "Chrome",  
    "deviceName": "Samsung Galaxy S23",  
    "platformName": "Android"  
}
```

The automation logs into the mobile application using the same tenant credentials and searches for the created project.

The test validates that the project appears correctly and that the mobile interface displays all relevant project information.

Executing the same business workflow across desktop and mobile devices increases confidence in cross-platform compatibility.

9. Tenant Isolation Validation

One of the most important requirements for a multi-tenant SaaS platform is ensuring that one organization cannot access another organization's data.

After confirming that the project is visible to Company 1, the automation logs out and signs in using credentials belonging to Company 2.

The automation then searches for the previously created project.

```
expect(  
    page.locator(f"text={project_name}")  
).not_to_be_visible()
```

If the project is visible to another tenant, it indicates a serious security vulnerability.

This validation confirms that tenant isolation has been implemented correctly.

10. Cleanup Strategy

Automation should always restore the environment to its original state.

Once all validations are complete, the project should be deleted through the API.

```
requests.delete(  
    f"https://app.workflowpro.com/api/v1/projects/{project_id}",  
    headers=headers  
)
```

Cleanup should execute even if the test fails.

Therefore, the deletion logic should be placed inside a finally block or implemented using Pytest fixtures.

Automatic cleanup prevents the accumulation of unnecessary test data in shared environments.

11. Complete Integration Test

```
import uuid  
  
import requests  
  
from playwright.sync_api import expect  
  
def test_project_creation_flow():
```

```
project_name = f"Automation_Project_{uuid.uuid4().hex[:8]}"
```

```
headers = {  
    "Authorization": f"Bearer {TOKEN}",  
    "X-Tenant-ID": COMPANY1_ID  
}
```

```
response = requests.post(  
    "https://app.workflowpro.com/api/v1/projects",  
    headers=headers,  
    json={  
        "name": project_name,  
        "description": "Automation Test",  
        "team_members": []  
    }  
)
```

```
assert response.status_code == 201
```

```
project_id = response.json()["id"]
```

```
try:
```

```
# Web UI Validation
```

```
page.goto(BASE_URL)
```

```
login_page.login(EMAIL, PASSWORD)
```

```
dashboard_page.verify_project(project_name)
```

```
# Mobile Validation (BrowserStack)
```

```
mobile_dashboard.verify_project(project_name)
```

```
# Tenant Isolation
```

```
logout()
```

```
login_page.login(COMPANY2_EMAIL, COMPANY2_PASSWORD)
```

```
dashboard_page.verify_project_not_present(project_name)
```

```
finally:
```

```
requests.delete(  
    f"https://app.workflowpro.com/api/v1/projects/{project_id}",  
    headers=headers  
)
```

This implementation demonstrates how API testing, Playwright automation, BrowserStack mobile validation, and security verification can be combined into a single end-to-end integration test.

12. Handling Edge Cases

A robust integration test should also consider situations where unexpected failures occur.

If the API returns a server error or an invalid response, the test should fail immediately because subsequent UI validations would not be meaningful.

If network latency delays project creation, the UI should wait until the project becomes visible instead of relying on fixed delays.

If BrowserStack experiences temporary connectivity issues, retry mechanisms can be used selectively for infrastructure-related failures.

The framework should also capture screenshots, browser traces, execution logs, and videos whenever a failure occurs to simplify debugging.

Finally, cleanup should always execute, regardless of whether the test passes or fails, ensuring that the testing environment remains clean for future executions.

13. Assumptions

The following assumptions were made while designing the integration test.

- API authentication tokens are available before execution.
- Dedicated automation users exist for each tenant.
- The application provides stable UI locators for project verification.
- BrowserStack credentials are configured externally using environment variables.
- The API supports project deletion after execution.
- Test environments allow automatic creation and removal of test data.
- Tenant databases are completely isolated.
- Mobile and web applications access the same backend database.

14. Conclusion

The proposed integration test validates the complete business workflow of project creation across API, web, and mobile layers while ensuring tenant isolation and proper cleanup. By using API calls for test setup, Playwright for web validation, BrowserStack for cross-platform testing, and automated cleanup, the solution achieves faster execution, improved reliability, and better maintainability than a UI-only approach.

This strategy aligns with enterprise automation best practices and provides high confidence that the application's core functionality works correctly across different environments, browsers, devices, and tenant configurations.

